

ANDROID STUDIO

HELBURUAK

Gai honetan kontzeptu hauek landuko ditugu:

1. Ingurunearen instalazioa (Android Studio)
2. Lehenengo proiektua sortu.
3. Proiektu baten estruktura
4. Aplikazio baten osagaiak
5. MainActivity aztertzen

INSTALAZIOA

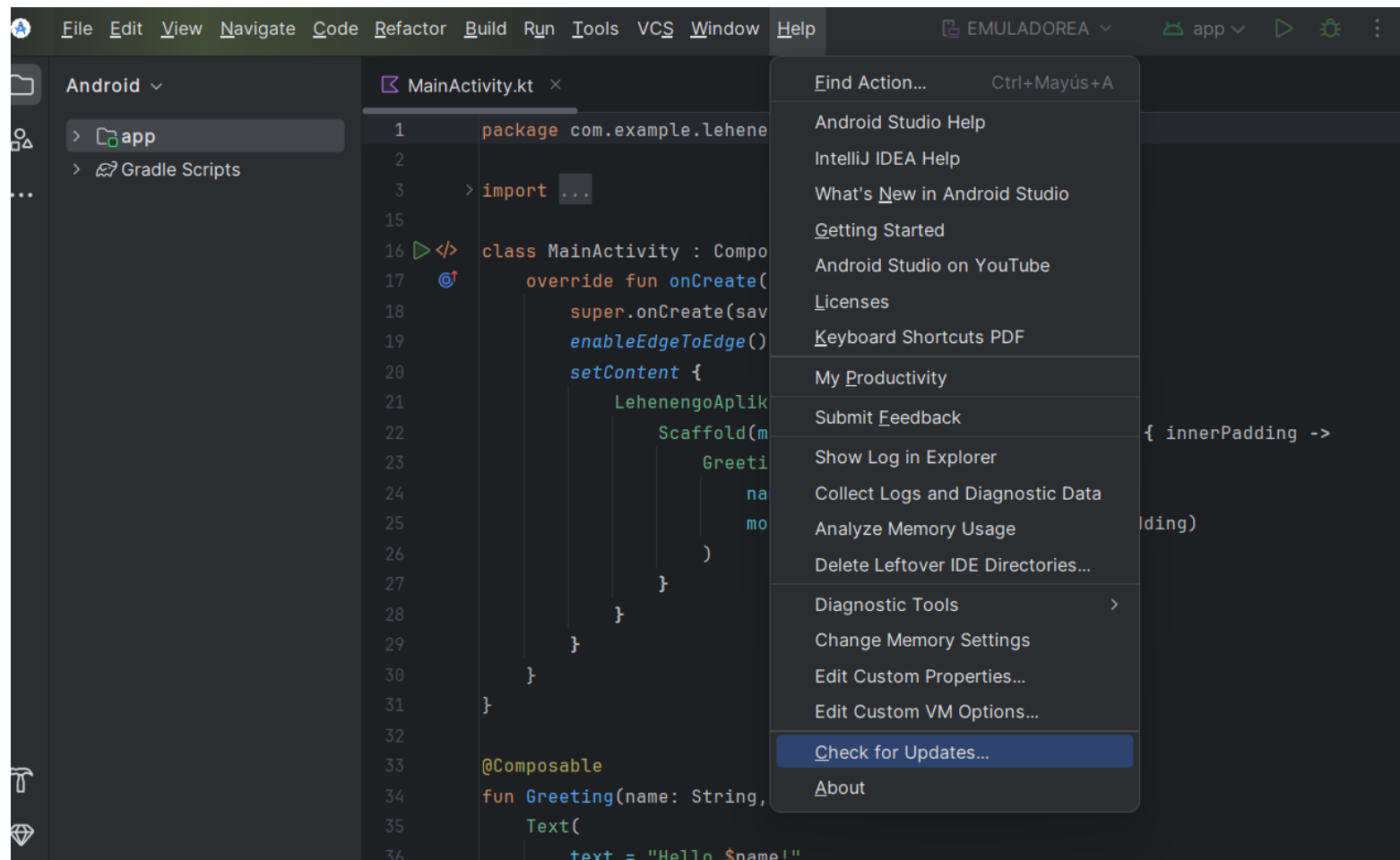
Lehenik eta behin Android Studioren azken bertsiko egonkorra instalazioa burutuko dugu:

<https://developer.android.com/studio>

Instalatuta bazenuen, begiratu azken bertsioa duzun:

- ▶ Sakatu **Help** --> **Check for Updates...**
Eskaintzen dizun eguneraketak burutu.

INSTALAZIOA



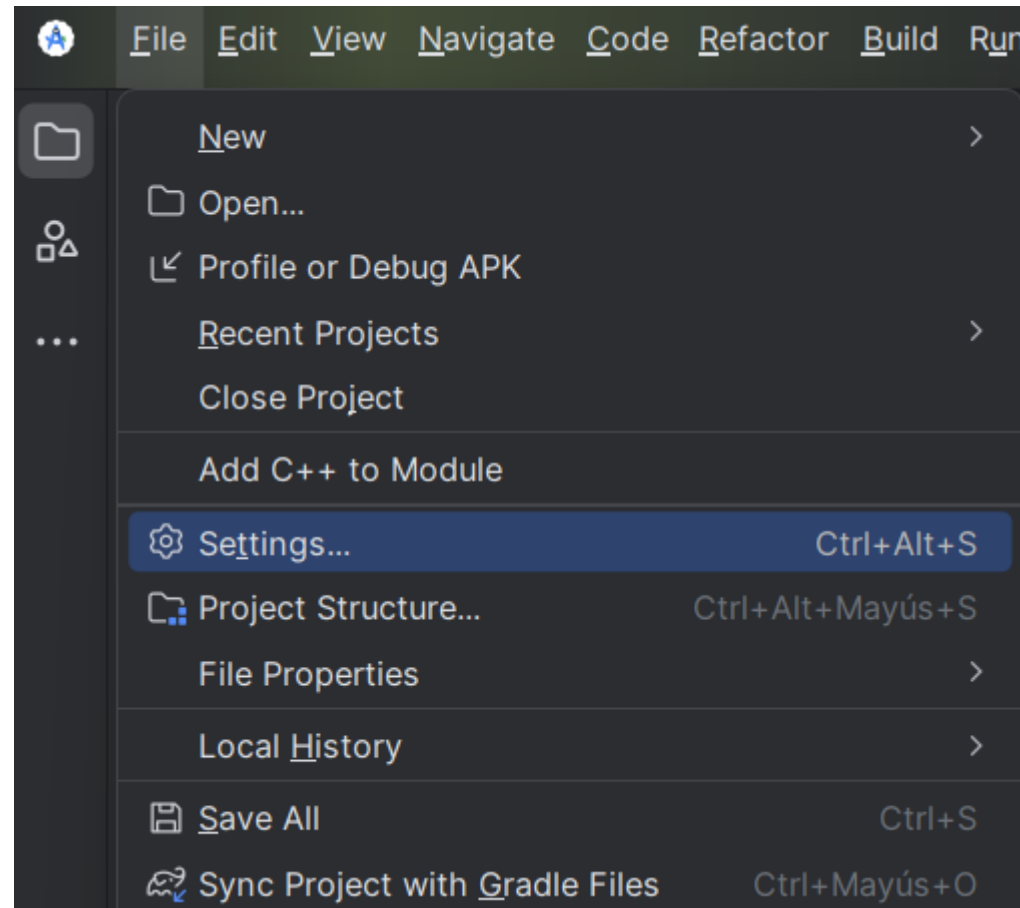
INSTALAZIOA

Android Studio 4.1 bertsiotik aurrera Kotlin plugina instalatuta agertzen da jada, beraz, ez dugu eskuz instalatu behar.

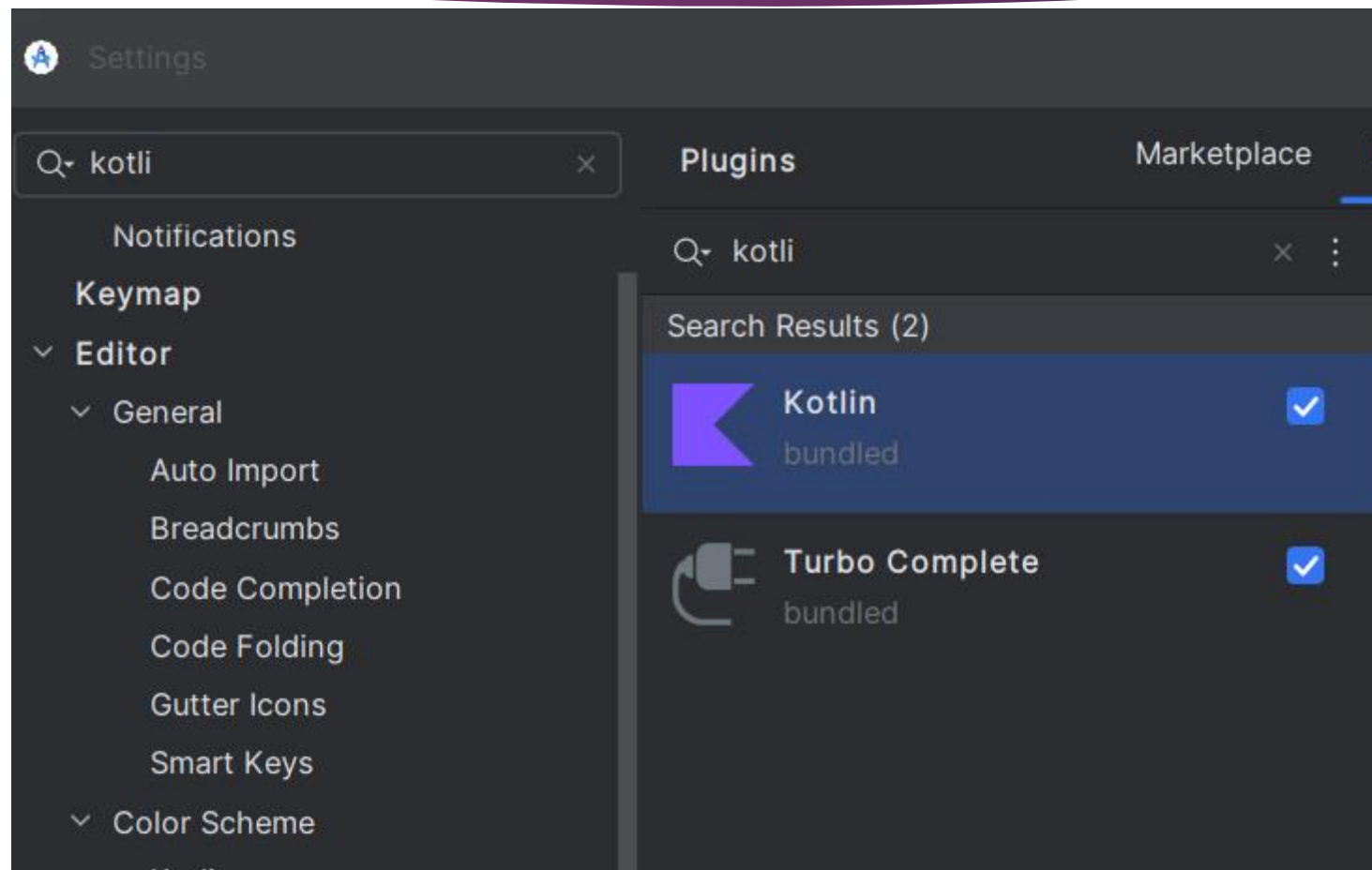
Azken bertsioa instalatuta duzula ziurtatzeko, joan Android Studioko plugin-konfigurazioen atalera. Horretarako:

- ▶ Sakatu **File** → **Settings...**
- ▶ Ondoren, bilatu **Plugins** alboko menuan eta Idatzi "Kotlin" goiko bilaketa-kutxan. Egiaztatu "Update" botoia agertzen den ala ez eta horrela bada sakatu

INSTALAZIOA



INSTALAZIOA



PROIEKTU BERRIA SORTU

Oso erraza da. Android Studio 3.0 bertsiotik aurrera, Kotlin erabat integratuta dago ingurunean, eta, beraz, zuzenean sor ditzakezu zure proiektuak Kotlinen. Ez du inolako misteriorik.

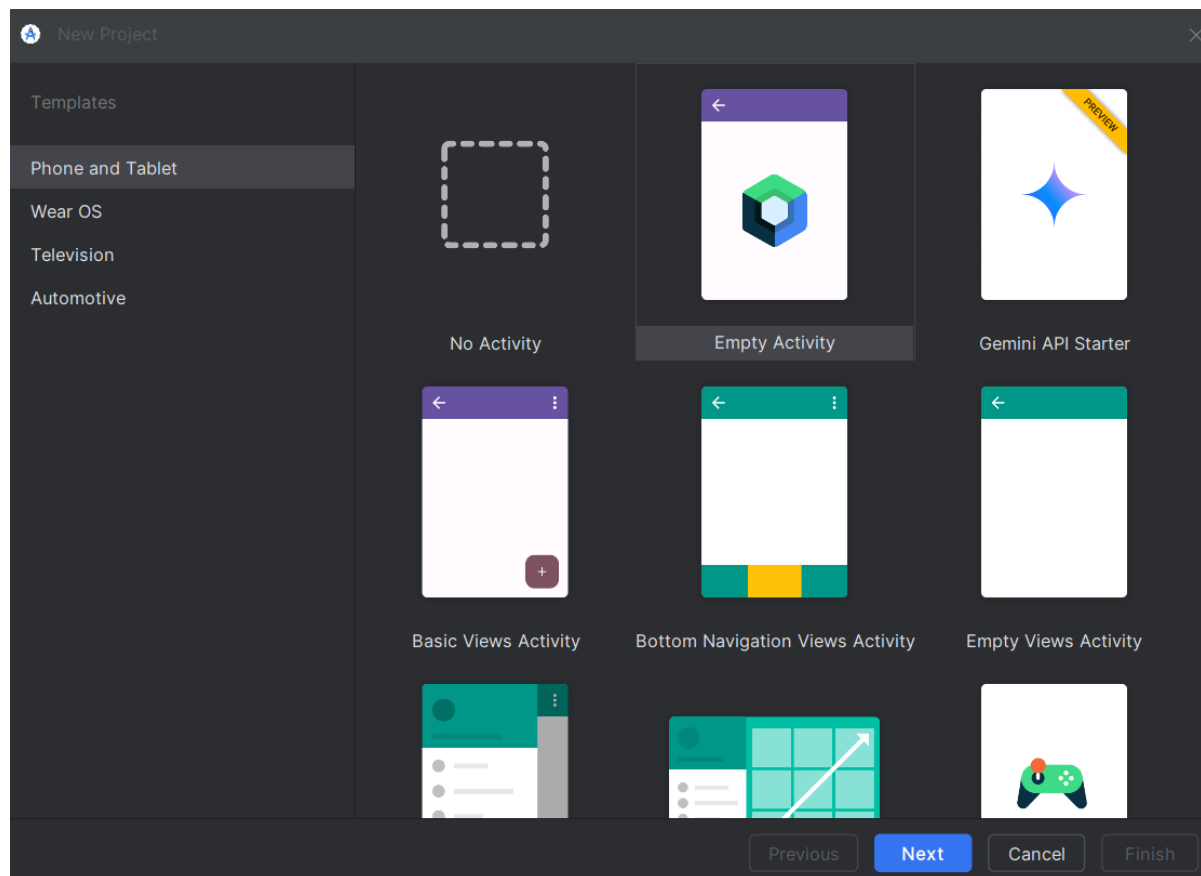
Hala ere, ikusiko dugu nola konfiguratu eskuz, oso erabilgarria izango dena Javan proiektu bat baduzu eta bertan Kotlin erabiltzen hasi nahi baduzu.

► Sakatu **File** → **New** → **New Project**

Pantaila hau agertuko zaizu:

PROIEKTU BERRIA SORTU

Empty Activity
aukeratueta
Hurrengoa.



PROIEKTU BERRIA SORTU

Pantaila honetan zure proiektua konfiguratuko da.

- 1.- Adierazi nahi duzun izena eta kokapena.
- 2.- Androiden bertsio minimoa ere nahi duzuna izan daiteke. Oro har, erabiltzaileen % 85 inguru bada, zifra ona da.
- 3.- Gainerakoa lehenetsita utz dezakezu.
- 4.- Sakatu Finish.

Oharra: Java hizkuntzan duzun proiekturen bat baduzue Java aukeratu eta bihurketa egin. Bestela kotlin aukeratu.

PROIEKTU BERRIA SORTU

New Project

Empty Activity

Create a new empty activity with Jetpack Compose

Name

Package name

Save location

Minimum SDK

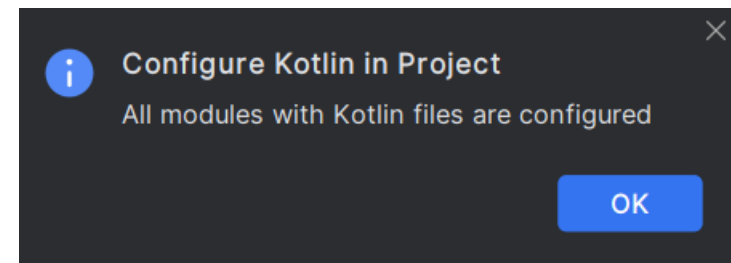
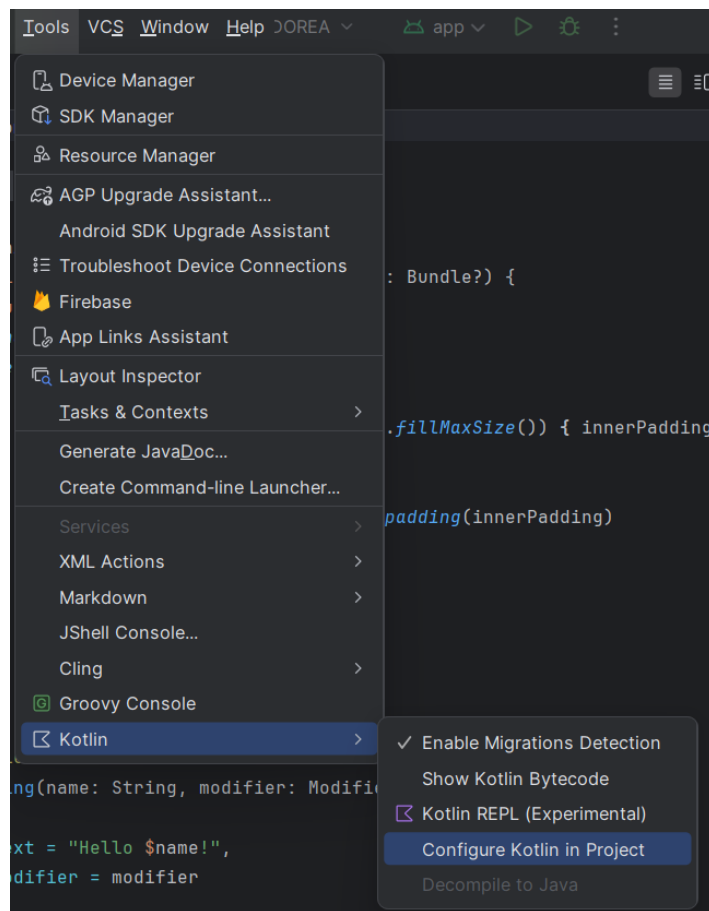
 Your app will run on approximately 97,4% of devices.
[Help me choose](#)

Build configuration language 

Previous Next Cancel Finish

PROIEKTU BERRIA SORTU

Kotlin-en konfigurazioa
begitu nahi izanez gero:



PROIEKTU BATEN ESTRUKTURA

Proiektua fitxategi eta karpeta osatua dago:

Moduloa

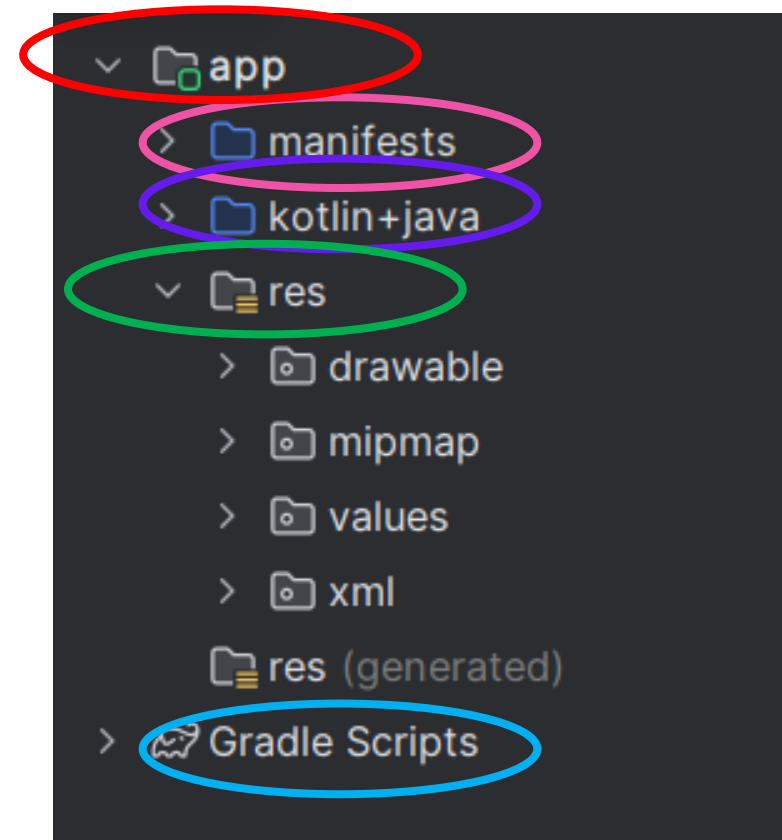
Aplikazioaren deskribatzailea

Kodigoa

Errekurtsoak

Konpilazioko konfigurazioa

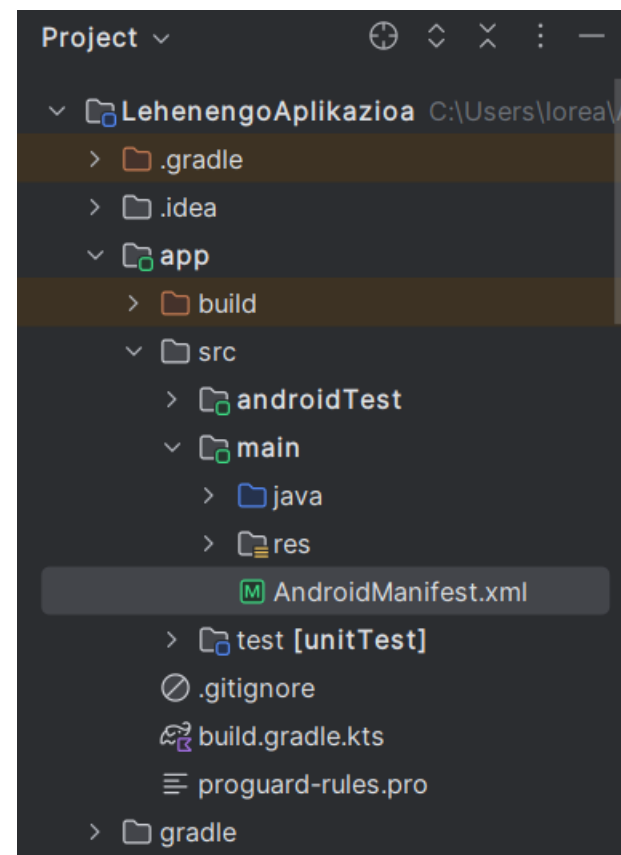
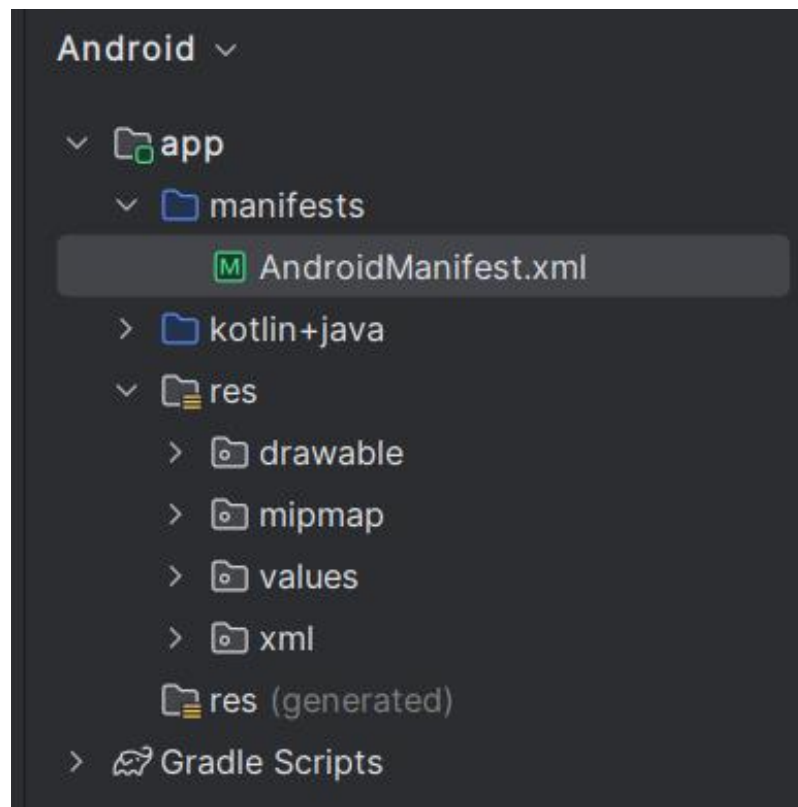
Gomendioa Project erabiltzea



PROIEKTU BATEN ESTRUKTURA

AndroidManifest.xml: Android aplikazioa deskribatzen du. Bertan, aplikazioaren jarduerak, barneratzeak, zerbitzuak eta edukiaren hornitzaileak adierazten dira. Halaber, aplikazioak eskatzen dituen baimenak deklaratzeko dira. Androiden bertsio minimoa adierazten da exekutatu ahal izateko, Java paketea, aplikazioaren bertsioa, etab.

PROIEKTU BATEN ESTRUKTURA



PROIEKTU BATEN ESTRUKTURA

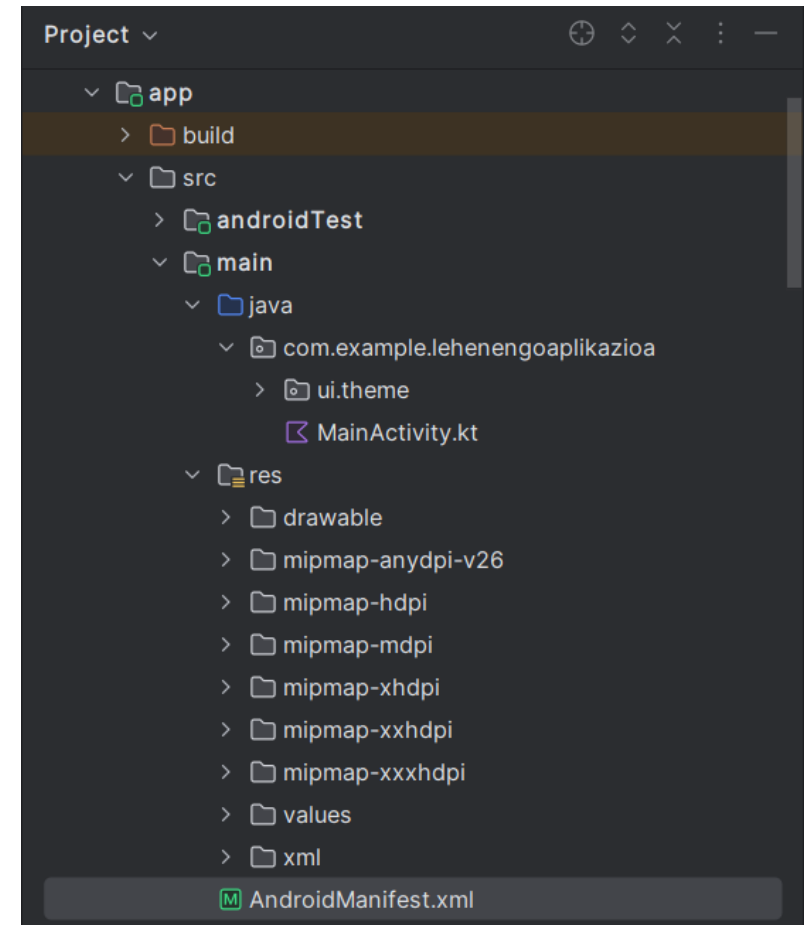
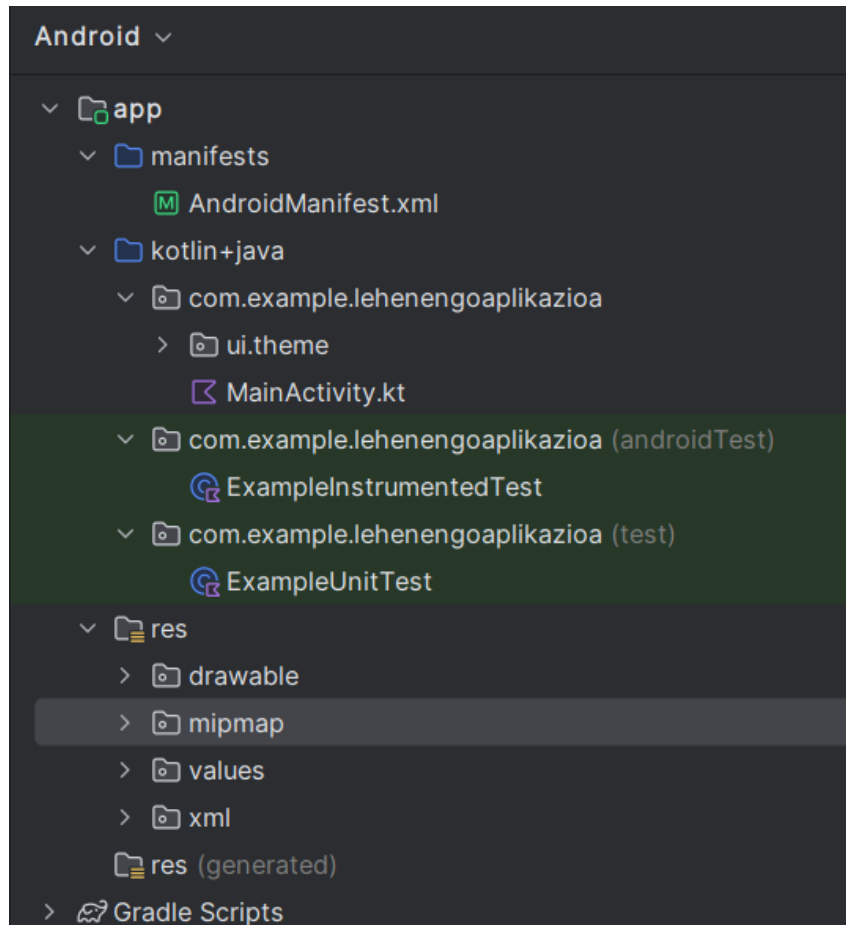
```
MainActivity.kt  AndroidManifest.xml ×
2      <manifest xmlns:android="http://schemas.android.com/apk/res/android"
5          <application
7              android:icon="@mipmap/ic_launcher"
10             android:label="LehenengoAplikazioa"
11             android:roundIcon="@mipmap/ic_launcher_round"
12             android:supportsRtl="true"
13             android:theme="@style/Theme.LehenengoAplikazioa"
14             tools:targetApi="31">
15         <activity
16             android:name=".MainActivity"
17             android:exported="true"
18             android:label="LehenengoAplikazioa"
19             android:theme="@style/Theme.LehenengoAplikazioa">
20             <intent-filter>
21                 <action android:name="android.intent.action.MAIN" />
22
23                 <category android:name="android.intent.category.LAUNCHER" />
24             </intent-filter>
25         </activity>
26     </application>
27
28 </manifest>
```


PROIEKTU BATEN ESTRUKTURA

Java: aplikazioaren iturburu-kodea duen karpeta. Paketearen izenaren arabera gordetzen da karpetetan.

- ▶ **MainActivity:** Java klasea hasierako jardueraren kodearekin.
- ▶ **ApplicationTest:** JUnit API erabiliz aplikazioaren testatze-kodea txertatzeko pentsatutako Java klasea.

PROIEKTU BATEN ESTRUKTURA

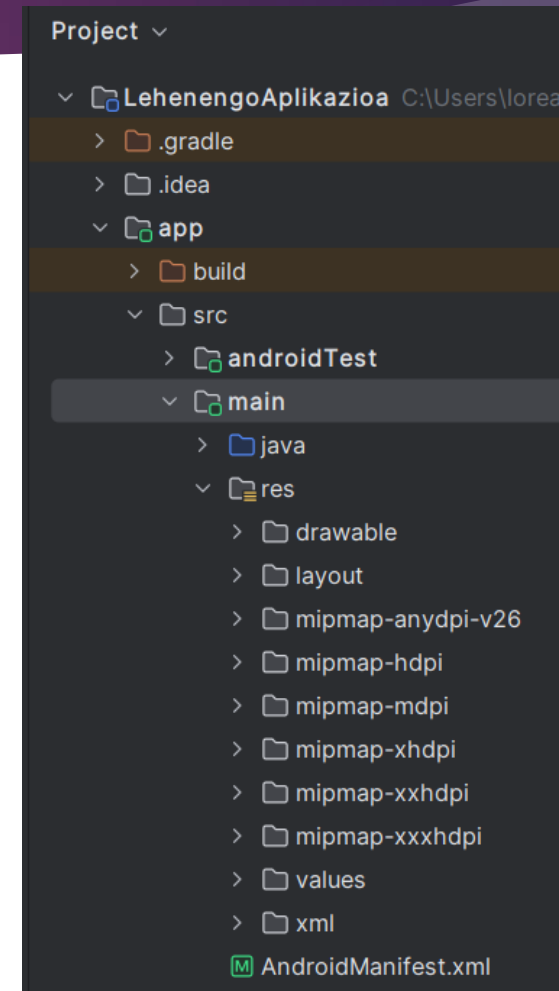
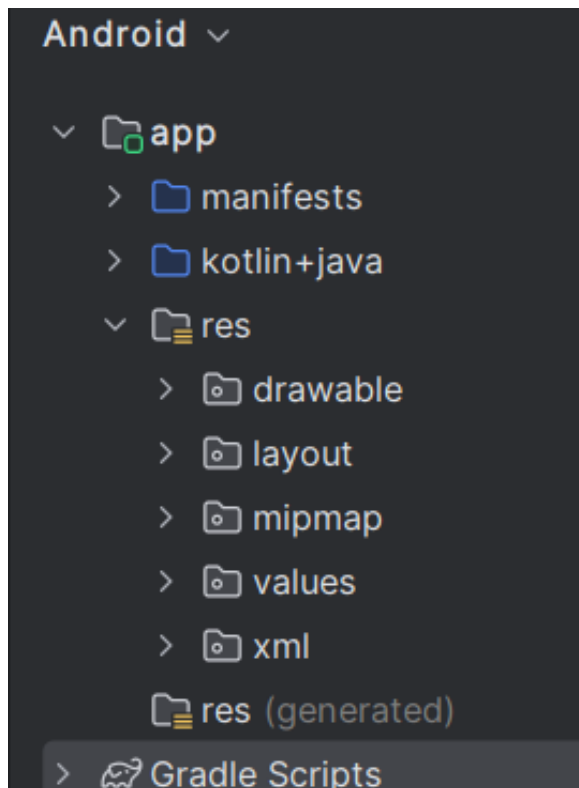


PROIEKTU BATEN ESTRUKTURA

Res: Aplikazioak erabiltzen dituen baliabideak biltzen dituen karpeta.

- ▶ **Drawable:** karpeta horretan gordetzen dira irudi-fitxategiak (jpg edo png) eta irudi-deskribatzaileak XMLn (atzizki berezi bat erabiliko dugu baliabide baten bertsio bat baino gehiago izan nahi badugu).
- ▶ **Mipmap:** drawableak bezala, baina sistemek ezin dituzte berriro eskalatu
- ▶ **Layout:** Aplikazioaren bistako xml fitxategiak ditu. Bisten bidez, aplikazioaren erabiltzaile-interfazea osatuko duten pantailak konfiguratu ahal izango ditugu. Web-orrialdeak diseinatzeko erabilitako HTMLaren antzeko formatu bat erabiltzen da.

PROIEKTU BATEN ESTRUKTURA

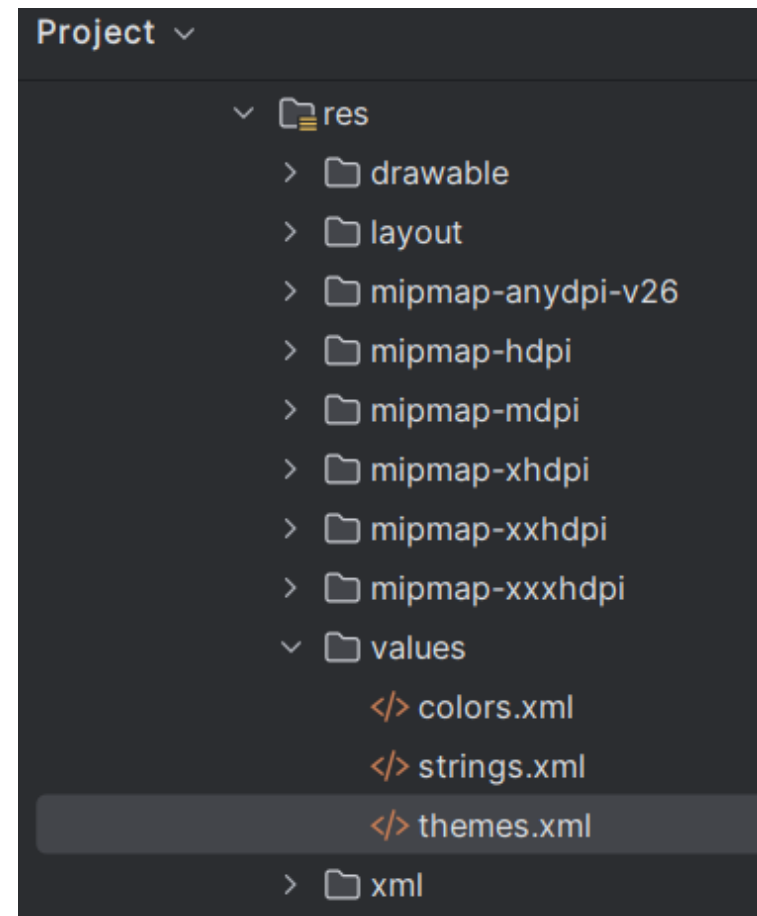
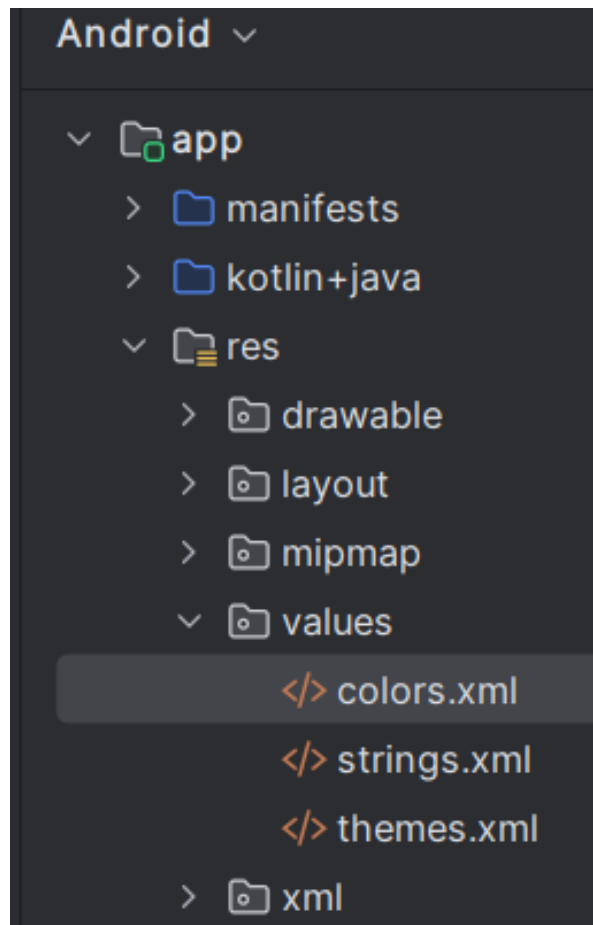


PROIEKTU BATEN ESTRUKTURA

- ▶ **Menu:** XML fitxategiak jarduera bakoitzeko menuekin.
- ▶ **Values:** XML fitxategiak ere erabiliko ditugu aplikazioan erabilitako balioak adierazteko. Horrela, fitxategi horietatik aldatu ahal izango ditugu iturburu-kodera joan barik.
 - ▶ **colors.xml** aplikazioan erabiliko diren koloreak definituko dira.
 - ▶ **string.xml** fitxategian, aplikazioaren karaktere-kate guztiak definitzen dira.
 - ▶ **themes.xml**, aplikazioaren estiloak eta gaiak definitzen dira.

Baliabide alternatiboak sortuz, oso erraza izango da aplikazio bat beste hizkuntza batera itzultzea.

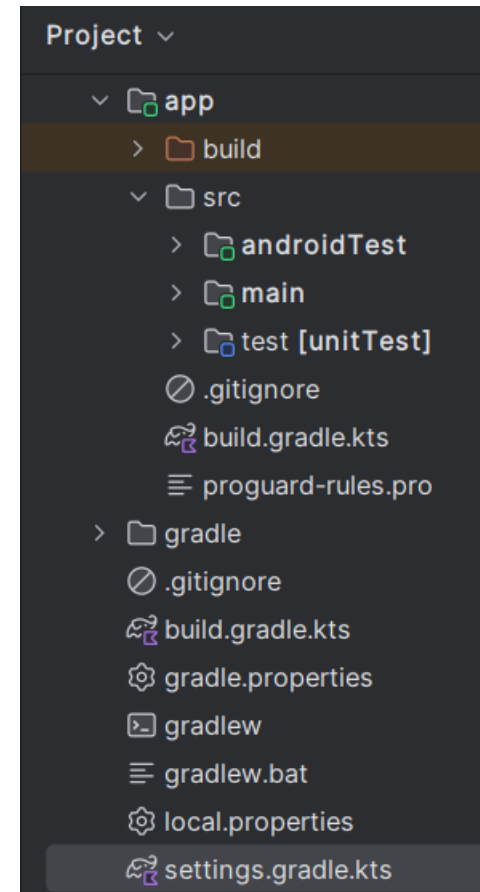
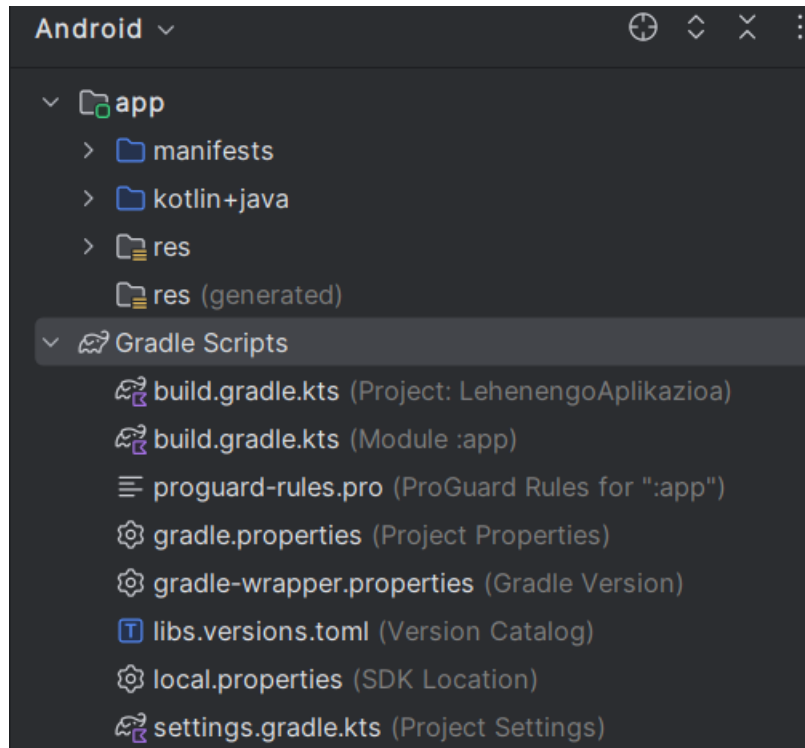
PROIEKTU BATEN ESTRUKTURA



PROIEKTU BATEN ESTRUKTURA

Gradle Scripts: Karpeta honetan Gradle fitxategi batzuk gordetzen dira aplikazioa eraikitzeko. Zenbait alderdi garrantzitsu defini ditzakezue, hala nola konpilazioaren sdk bertsioa (targetSdkVersion) eta gutxieneko bertsioa (minSdkVersion).

PROIEKTU BATEN ESTRUKTURA



PROIEKTU BATEN ESTRUKTURA

Laburbilduz proiektuaren barruan, honako hauek aurkituko ditugu:

- ▶ **Jarduera nagusia:** Jarduera bat sortzen da, erabiltzaileari erakutsiko zaion lehena.
- ▶ **Ikusmenaren definizioa:** layout-aren barruan egotea
- ▶ **Baliabideen karpeta:**
 - ▶ **drawable:** irudi-fitxategiak eta irudi-deskribatzaileak (.png, .jpg, .xml)
 - ▶ **mipmap:** drawable bezala, baina sistemak ezin ditu berriz eskalatu,
 - ▶ **layout:** XML fitxategiak aplikaziora begira. Aplikazioaren pantailak diseinatzeko aukera ematen du.

PROIEKTU BATEN ESTRUKTURA

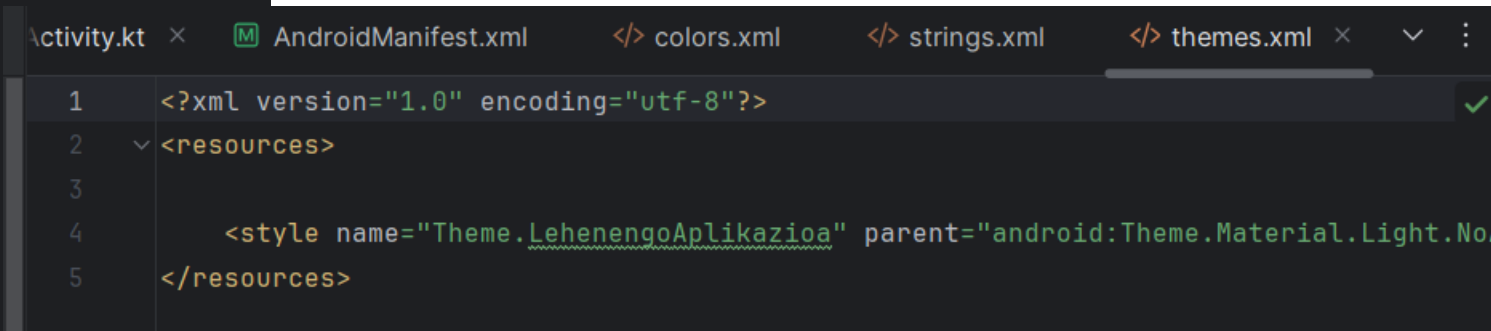
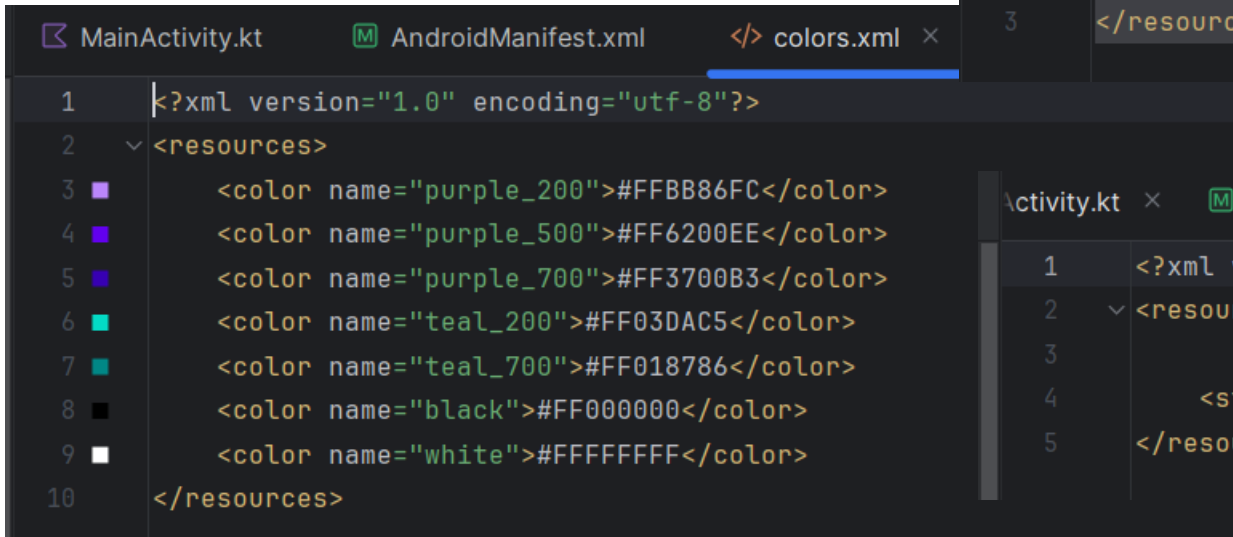
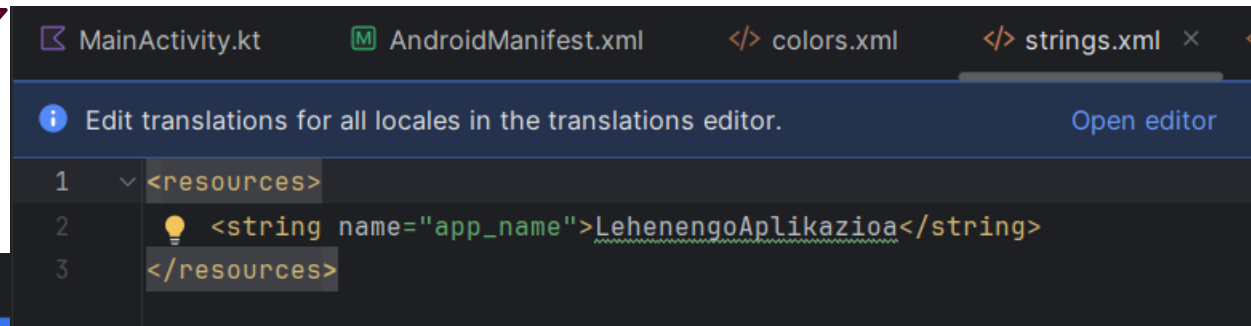
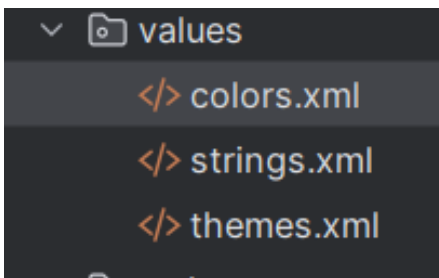
- ▶ **value:** kate-balioak, koloreak eta estiloak...
- ▶ **menua:** XML fitxategiak, aplikazioaren menuekin.
- ▶ **anim:** animazio-deskribatzaileak dituzten XML fitxategiak.
- ▶ **xml:** aplikazioak eskatzen dituen beste XML fitxategi batzuk.
- ▶ **raw:** XML formatuan ez dauden fetxero gehigarriak
- ▶ **doc:** proiektuari lotutako dokumentazioa

PROIEKTU BATEN ESTRUKTURA

Balioen definizioa

- ▶ Balioa magnitude bat definitzen duen baliabidea da
- ▶ Zenbait kolore, testu-kate, estilo, dimentsio eta abarretan definitzen dira
- ▶ Kodea eta edukia bereizteko aukera ematen du
- ▶ Gainera aplikazioa konfiguratzea eta itzultzea errazten du

PROIEKTU BATEN ESTRUKTURA



APLIKAZIO BATEN OSAGAIAK

Jarduera (Activity)

Androideko aplikazio bat, bistaratzeko oinarrizko elementu multzo batez osatuta egongo da, aplikazioaren **pantailak** izenez ezagutzen direnak. Android sisteman, elementu edo pantaila horietako bakoitzari **jarduera** esaten zaio. Haren funtzio nagusia erabiltzaile-interfazea sortzea da. Aplikazio batek hainbat jarduera behar izaten ditu erabiltzaile-interfazea sortzeko. Sortutako jarduerak independenteak izango dira, nahiz eta guztiek helburu komun baterako lan egin.

APLIKAZIO BATEN OSAGAIAK

Bistak

Aplikazio baten erabiltzaile-interfazea osatzen duten elementuaz osatuak daude. (Adibidez, botoia edo testu-sarrera bat) view klasearen ondorengoak dira eta, beraz, Java kodea erabiliz defini daitezke. Hala ere, ohikoena izango da bistak XML fitxategia erabiliz definitzea, eta sistemak fitxategi horretatik abiatuta guk nahi ditugun objektuak sortzea.

Layout

Forma jakin bateko ikuspegi taldekatuen multzoa da. Hainbat layout-mota izango ditugu forma linealeko bistak antolatzeko, koadrikulatan edo bista bakoitzaren posizio absolutua adieraziz.

APLIKAZIO BATEN OSAGAIAK

Zerbitzua (service)

Erabiltzailearekin interakziorik egin beharrik gabe gauzatzen den prozesua da. Unix-eko deabru baten edo Windows-eko zerbitzu baten antzeko zerbait da. Androietan, bi motatako zerbitzuak ditugu: zerbitzu lokalak, prozesu berean exekutatzen direnak, eta urruneko zerbitzuak, prozesu bereizian(separados) exekutatzen direnak.

APLIKAZIO BATEN OSAGAIAK

Asmoa (Intent)

Ekintzaren bat egiteko aukera adierazten du; adibidez, telefono-dei bat egitea edota web-orri bat ikustea. Jarduera bat abiarazi, zerbitzu bat abiarazi, broadcast motako iragarki bat bidali edo zerbitzu batekin komunikatu nahi dugun bakoitzean erabiltzen da. Bidalitako osagaiak gure aplikazioaren barrukoak edo kanpokoak izan daitezke. Osagai horien artean informazioa trukatzeko ere erabiliko ditugu.

APLIKAZIO BATEN OSAGAIAK

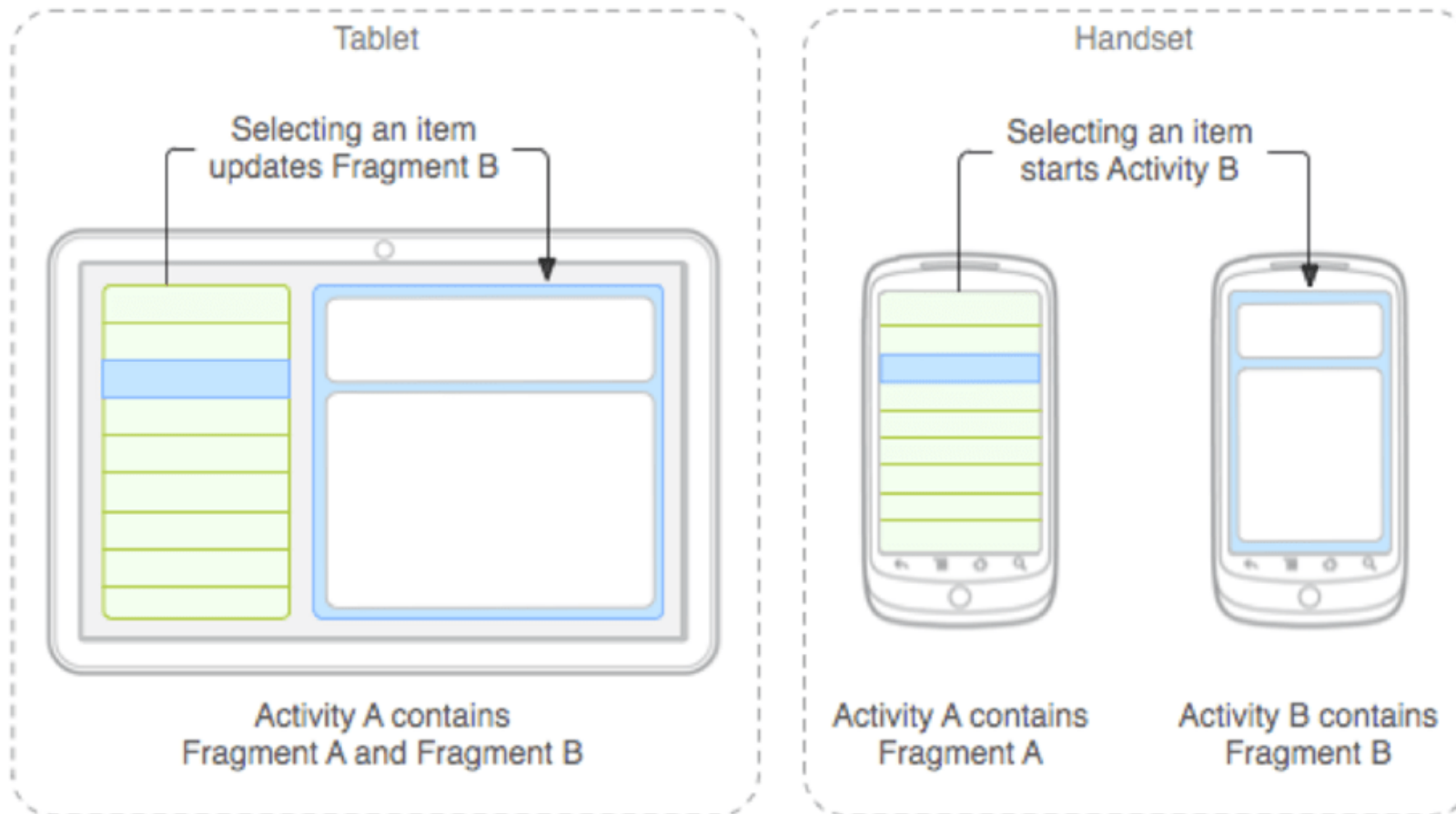
Fragment

Tableten etorrerak arazoa berri bat ekarri zuen, android aplikazioek pantaila handiagoak jasan behar zituen. Gailu mugikor baterako pentsatutako aplikazio bat diseinatzeko badugu eta gero tableta batean exekutatzeko badugu, emaitza ez da aproposa izango.

Diseinatzaileari arazo hau konpontzen laguntzeko, Android-en 3.0 bertsioan fragmentak agertzen dira. Fragment bat hainbat bisten loturak osatzen du, erabiltzaile-interfazearen bloke funtzional bat sortzeko. Fragmentak sortu ondoren, jarduera(Activity) baten barruan fragment bat edo gehiago konbina ditzakegu, pantailaren tamainaren arabera.



APLIKAZIO BATEN OSAGAIK



APLIKAZIO BATEN OSAGAIAK

Iragarkien hartzailea (Broadcast receiver)

Iragarkien hargailu batek broadcast motako iragarkien aurrean jaso eta erreakzionatzen du. Broadcast iragarkiak sistemaren edo aplikazioen arabera antola daitezke. Sistemak sortutako iragarki mota batzuk honako hauek dira: bateria baxua, sarrera deia... Aplikazioek broadcast iragarki mota berriak sortu eta merkaturatu ditzakete. Iragarkien hartzaileek ez dute erabiltzaile-interfazerik, baina jarduera bat has dezakete egokitzen jotzen badute.

APLIKAZIO BATEN OSAGAIAK

Eduki-hornitzaileak(Content Provider)

Askotan, Android terminal batean instalatutako aplikazioek informazioa partekatu behar izaten dute. Android-ek mekanismo estandar bat definitzen du, aplikazioek datuak partekatu ahal izan ditzaten fitxategi-sistemaren segurtasuna arriskuan jarri gabe. Mekanismo horren bidez, beste aplikazio batzuetako datuetara sartu ahal izango gara, hala nola kontaktu-zerrendara edo beste aplikazio batzuei datuak ematera.

MAINACTIVITY AZTERTZEN

Kode honetan automatikoki sortutako funtzio batzuk daude, bereziki, `onCreate ()` eta `setContent ()` funtzioak.

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            LehenengoAplikazioaTheme {
                Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
                    Greeting(
                        name = "Android",
                        modifier = Modifier.padding(innerPadding)
                    )
                }
            }
        }
    }
}
```

MAINACTIVITY AZTERTZEN

- ▶ **onCreate () funtzioa** da app honetako sarrera-puntua, eta beste funtzio batzuetara deitzen du erabiltzaile-interfazea konpilatzeko. Kotlinen programetan, main () funtzioa Kotlinen konpiladorea hasten den zure kodearen leku espezifikoa da. Androiderako app-etan, onCreate () funtzioak lan hori egiten du.
- ▶ **setContent () funtzioa**, onCreate () funtzioaren barruan, diseinua konposizioa onartzen duten funtzioen bidez definitzeko erabiltzen da. @Composable oharpenarekin markatutako funtzio guztiei setContent () funtziotik edo konposizioa onartzen duten beste batzuetatik dei dakieke. Oharrak Kotlineko konpiladoreari adierazten dio Jetpack Compos-ek funtzio hori erabiltzen duela IU sortzeko.

MAINACTIVITY AZTERTZEN

Jarraian, begiratu Greeting() funtzioa. Greeting () funtzioa bat dator konposagarritasunarekin. Begiratu haren gainean dagoen @Composable oharra. Konposagarritasun-funtzio batek sarreraren bat hartzen du eta pantailan erakusten dena sortzen du.

```
@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    Text(
        text = "Hello $name!",
        modifier = modifier
    )
}
```

MAINACTIVITY AZTERTZEN

Oraingoz, Greeting () funtzioak izen bat hartzen du eta pertsona horri zuzendutako Hello agurra erakusten du.

Eguneratu Greeting () funtzioa, "Hello" erakutsi beharrean ondorengoa aurkezteko(zuzenean eguneratuko da bestela  hurrengo botoia sakatu)

```
@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    Text(
        text = "Hello, my name is $name!",
        modifier = modifier
    )
}
```


MAINACTIVITY AZTERTZEN

Primeran! Testua aldatu duzue, baina Android gisa aurkezten zaituzte, ziurrenik zurea ez den izen bat. Ondoren, aurkezpena zure izenarekin pertsonalizatuko dugu.

`DefaultPreview ()` funtzioa bikaina da, eta aplikazioa erabat konpilatu gabe nola ikusten den ikusteko aukera ematen dizu. Aurretiko bistaren funtzioa izan dadin, `@Preview` ohar bat gehitu behar duzu.

Ikus dezakezunez, `@Preview` oharrak `showBackground` izeneko parametro bat hartzen du. `ShowBackground` egiazkotzat jotzen baduzu, atzealdea erantsiko zaio zure app-aren aurrebistari.

MAINACTIVITY AZTERTZEN

Android Studiok gai argi bat modu lehenetsian erabiltzen duenez editorearentzat, zaila izan daiteke showBackground = egiazkoa eta showBackground = faltsua arteko aldea ikustea. Hala ere, horrela ikusten da gai ilun batekiko desberdintasuna. Begiratu atzealde zuriari irudian, egiazkotzat jotzen denean.

```
@Preview(showBackground = true)
@Composable
fun GreetingPreview() {
    LehenengoAplikazioaTheme {
        Greeting( name: "Android")
    }
}
```

MAINACTIVITY AZTERTZEN

Eguneratu `DefaultPreview()` funtzioa zuen izenekin. Gero, konpilatu berriro app-a eta begiratu zure aurkezpen-txartel pertsonalizatua.

```
@Preview(showBackground = true)
@Composable
fun GreetingPreview() {
    LehenengoAplikazioaTheme {
        Greeting(name: "Lorea")
    }
}
```

GreetingPreview

Hello, my name is Lorea!

MAINACTIVITY AZTERTZEN

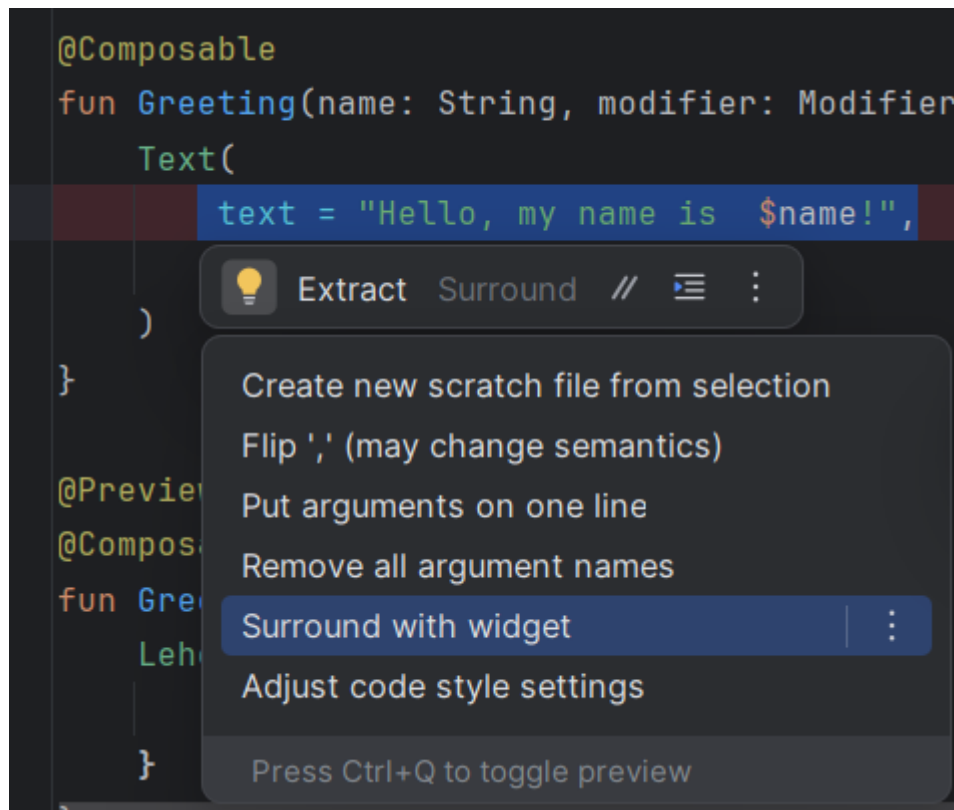
Aurkezpenerako atzealdeko kolore desberdina ezarri nahi baduzue, testua surface batean bildu beharko duzu. Surface bat IUren sekzio bat irudikatzen duen edukiontzi bat da, eta bertan itxura alda dezakezu, hala nola ertza edo hondoko kolorea.

Testua Surface batekin inguratzeko, testu-lerroa nabarmentzen da, presioa egiten du (**Alt+Enter** Windows-en edo **Option+Enter** Mac-en) eta, ondoren, **Surround with widget** hautatzen du.

MAINACTIVITY AZTERTZEN

```
@Composable
fun Greeting(name: String, modifier: Modifier
    Text(
        text = "Hello, my name is $name!",
    )
}

@Preview
@Composable
fun GreetingPreview() {
    Greeting("Leh")
}
```



The screenshot shows an IDE with a code snippet and a context menu. The code snippet is a Kotlin function 'Greeting' that takes 'name' and 'modifier' as parameters and returns a 'Text' widget with the text 'Hello, my name is \$name!'. The 'text' line is selected, and a context menu is open with options like 'Extract', 'Surround', and 'Surround with widget'.

MAINACTIVITY AZTERTZEN

Ezabatu `Box()` eta idatzi `Surface ()` bere orde. Surface edukiontzia koloretako parametro bat du. Konfiguratu kolore gisa. Ondorengo itzura izan behar du:

```
@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    Surface(color = Color){
        Text(
            text = "Hello, my name is $name!",
            modifier = modifier
        )
    }
}
```

MAINACTIVITY AZTERTZEN

Kolorea idazten duzunean, hitza gorriz eta azpimarratuta egongo da. Arazo hau konpontzeko, joan zaitez fitxategiaren goialderaino, han inportazioak azaltzen dira ondorengo bi import-ak gehitu.

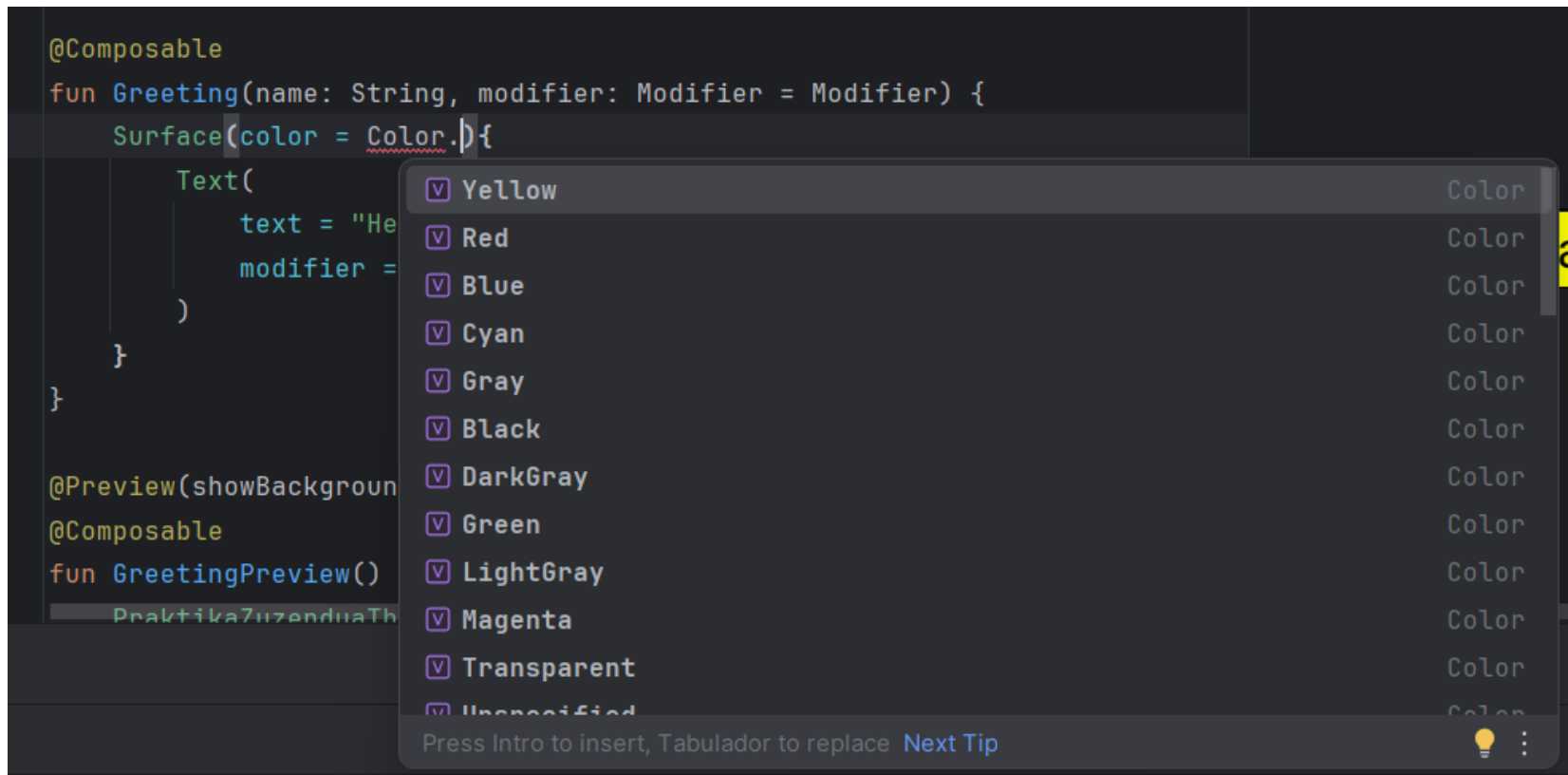
```
import androidx.compose.material3.Surface  
import androidx.compose.ui.graphics.Color
```

MAINACTIVITY AZTERTZEN

Aukeratu kolore bat gainazalerako:

```
@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    Surface(color = Color.) {
        Text(
            text = "He
            modifier =
        )
    }
}

@Preview(showBackground
@Composable
fun GreetingPreview()
    Praktika7uzenduaTh
```



The screenshot shows an IDE with a dark theme. A Kotlin Composable function `Greeting` is being edited. The `Surface` composable is being called with `color = Color.`, and a dropdown menu is open, showing a list of color options: Yellow, Red, Blue, Cyan, Gray, Black, DarkGray, Green, LightGray, Magenta, Transparent, and Unspecified. Each option has a small square icon to its left and the word 'Color' to its right. At the bottom of the dropdown, there is a hint: 'Press Intro to insert, Tabulador to replace' and a 'Next Tip' link. A lightbulb icon is also visible at the bottom right of the dropdown.

MAINACTIVITY AZTERTZEN

```
@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    Surface(color = Color.Magenta){
        Text(
            text = "Hello, my name is $name!",
            modifier = modifier
        )
    }
}
```

GreetingPreview

Hello, my name is Lorea!

MAINACTIVITY AZTERTZEN

Orain zuen testuak atzealdeko kolorea du; jarraian, espazio pixka bat (padding) gehituko dugu testuaren inguruan.

Modifier bat erabiltzen da konposizioa onartzen duen elementu bat handitzeko edo apaintzeko. Padding aldatzailea erabil dezakezu, elementuaren inguruan espazioa gehitzen duena (kasu honetan, testuaren inguruan). `Modifier.padding ()` funtzioaren bidez lortzen da hori.

Gogoratu gehitzeaz beharrezkoak diren inportazio deklarazioaren atalari. Eta nahi duzun tamainako padding aldatzaile bat gehitu testuari.

MAINACTIVITY AZTERTZEN

```
@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    Surface(color = Color.Magenta){
        Text(
            text = "Hello, my name is $name!",
            modifier = modifier.padding(30.dp)
        )
    }
}
```

GreetingPreview

Hello, my name is Lorea!